

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR AIRLINE RESERVATION AND FLIGHT OPERATIONS MANAGEMENT SYSTEM

DBMS Course Project | MySQL • Python • Streamlit

TEAM MEMBERS

Hrishika Birru • Sukriti • Mithila



Problem Description

Centralizing airline information requires reliable relationships, consistency and controlled access to data.

Passenger Management

Store and retrieve passenger details accurately.

Flight Operations

Manage flights, airports and aircraft information.

Reservations

Connect passengers with flights and reservation records.

Data Consistency

Prevent invalid or disconnected records through constraints.

Objectives & Proposed Solution

The system combines a relational database with a Python backend and user-facing Streamlit application.

Project Objectives

- Design a normalized relational airline database.
- Implement CRUD operations for core entities.
- Maintain integrity using PKs, FKs and constraints.
- Connect application logic to MySQL.
- Provide a simple, usable interface.

System Flow

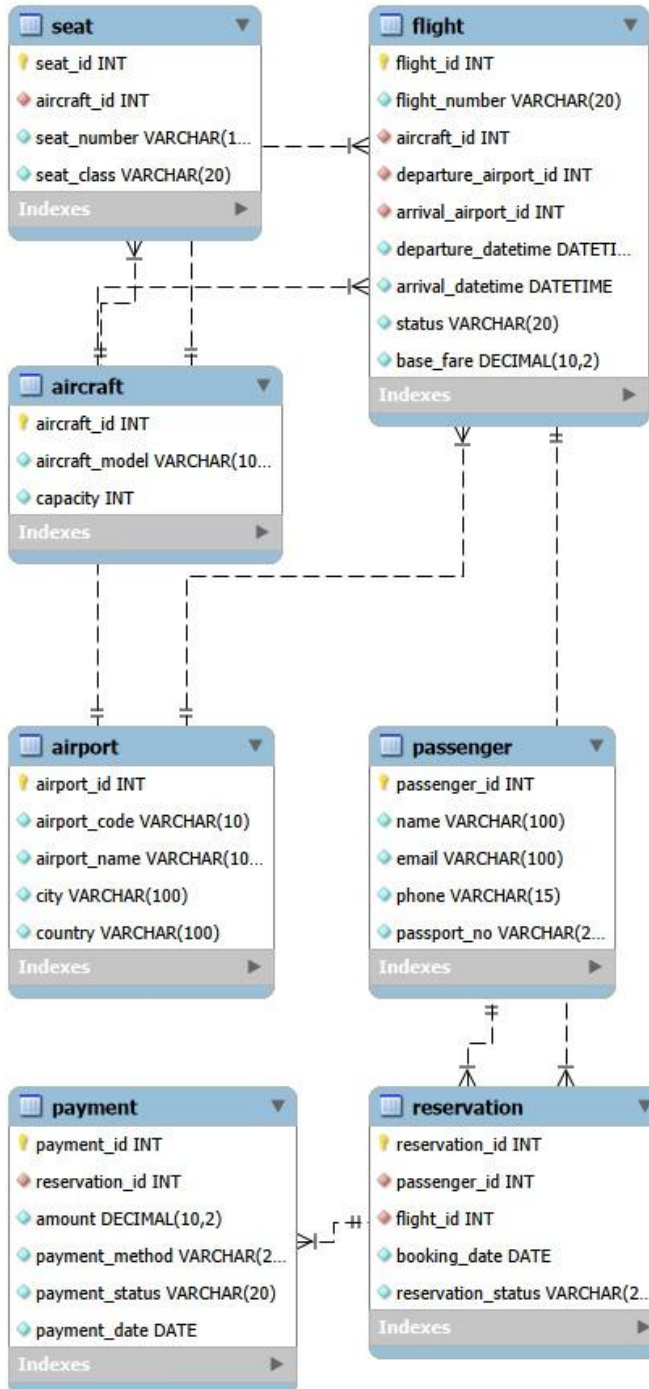


User action → backend logic → database operation

ER Diagram

Main Entities

- Passenger
- Airport
- Aircraft
- Flight
- Seat
- Reservation
- Payment



Database Schema

Core relations and representative attributes. Keep the final slide aligned with the actual SQL schema.

PASSENGER

passenger_id (PK)
name • email • phone
passport_no

AIRPORT

airport_id (PK)
airport_code • name
location

AIRCRAFT

aircraft_id (PK)
model • capacity

FLIGHT

flight_id (PK)
flight_number • aircraft_id (FK)
departure / arrival airport

RESERVATION

reservation_id (PK)
passenger_id (FK)
flight_id (FK)

PAYMENT

payment_id (PK)
reservation_id (FK)
amount • status

PK = Primary Key • FK = Foreign Key

Database Implementation

The relational model is implemented in MySQL with integrity constraints and linked tables.

MySQL Implementation

- Database: airline_reservation_db
- Core airline tables created in MySQL.
- Primary keys identify records uniquely.
- Foreign keys preserve relationships.
- NOT NULL / UNIQUE / CHECK constraints support validity.
- Referential integrity prevents invalid links.

Database Design Principles

IDENTITY

Primary keys

RELATIONSHIPS

Foreign keys

VALIDATION

Constraints

CONSISTENCY

Referential integrity

OPERATIONS

CRUD support

SQL Queries & Operations

Representative operations demonstrate how the database is created, read and modified.

```
SELECT * FROM Passenger;  
INSERT INTO Passenger  
(name, email, phone, passport_no)  
VALUES (...);
```

CRUD Operations

INSERT • SELECT • UPDATE • DELETE

Joins & Relationships

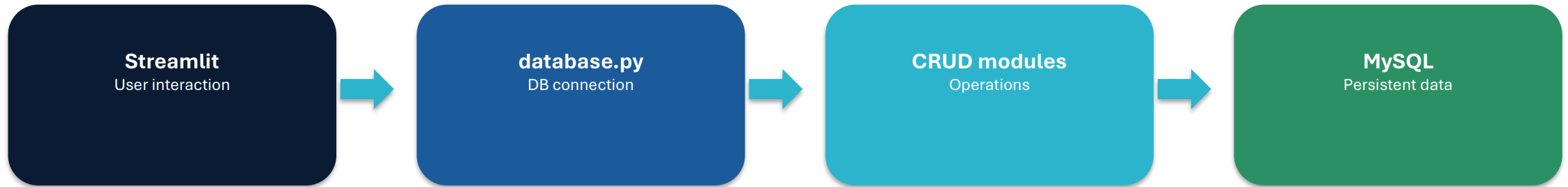
Queries can combine related passenger, flight and reservation records.

Constraint Enforcement

Database rules prevent invalid values and broken relationships.

Python Backend

The backend acts as the bridge between user actions and MySQL operations.



Backend Responsibilities

- Establish database connection
- Execute CRUD functions
- Validate inputs / handle errors
- Commit successful changes

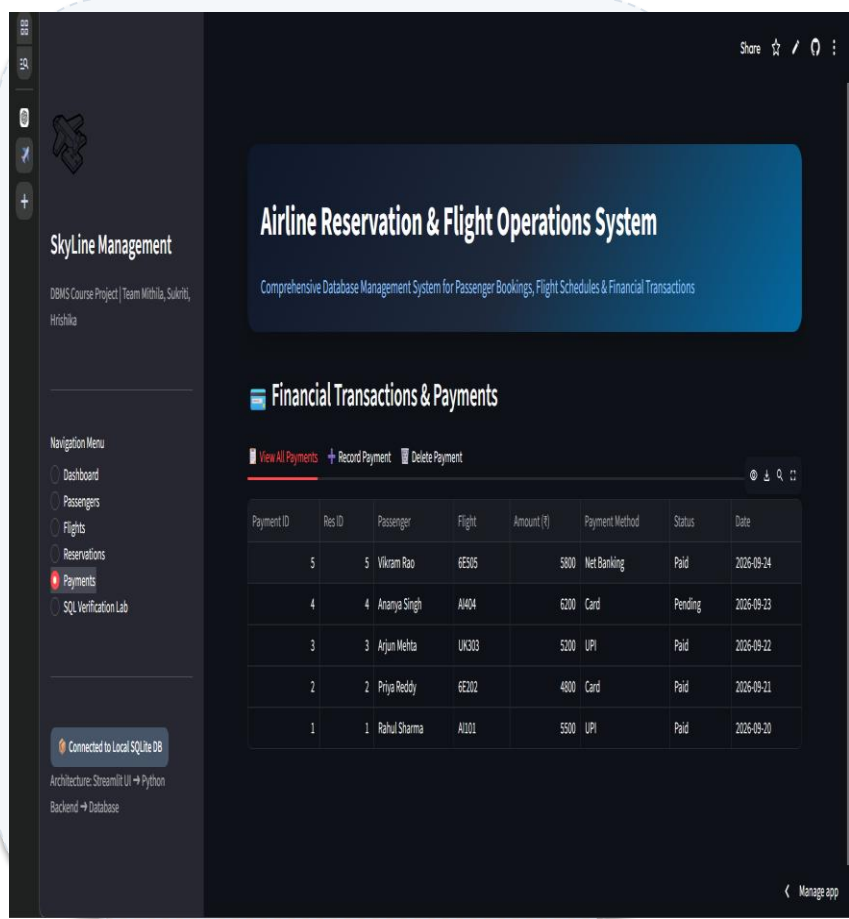
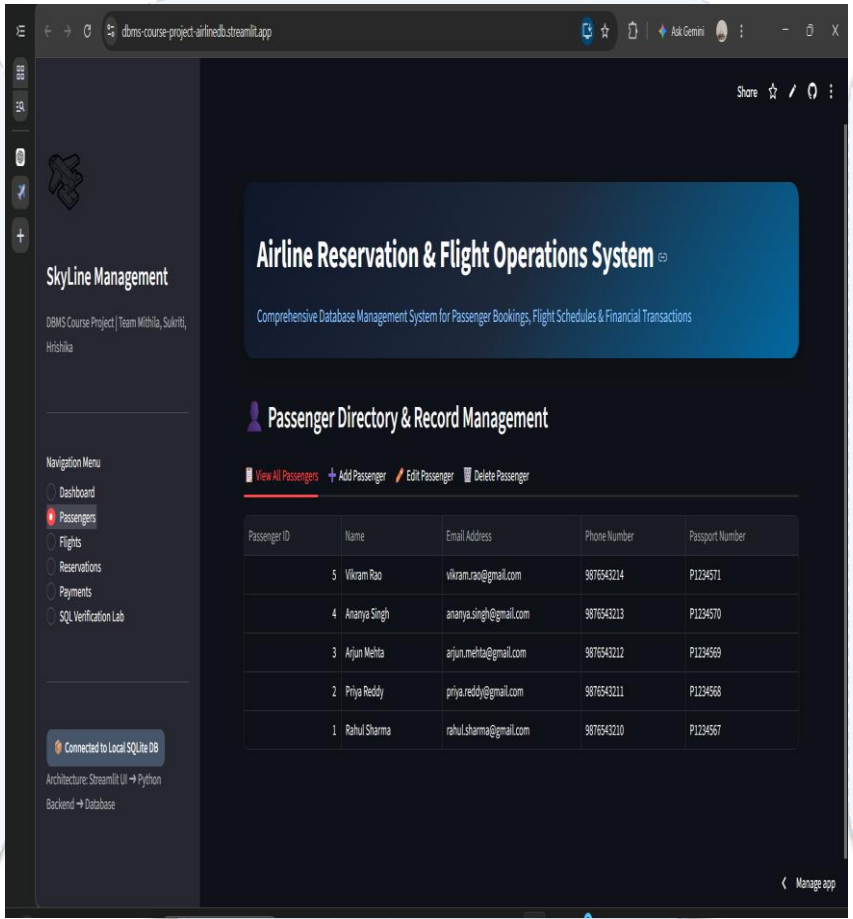
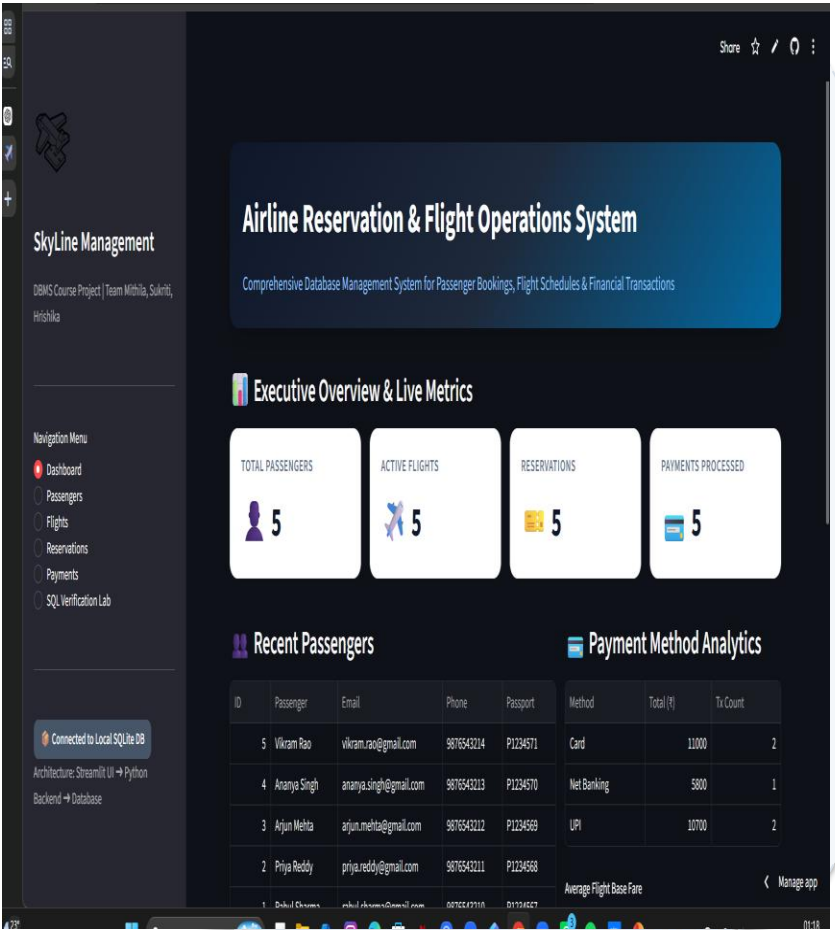
Project Modules

database.py
passenger.py
flight / reservation / payment modules
(Use your exact final module names.)

User Interface Demo

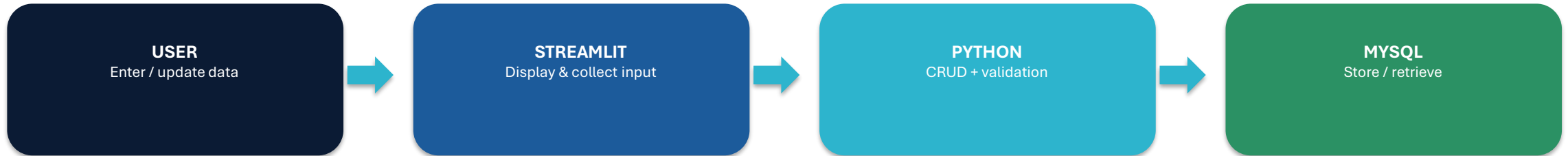
Show how users interact with the database through the Streamlit application.

<https://dbms-course-project-airlinedb.streamlit.app/>



Complete System Demo & Conclusion

End with the complete user-to-database flow, key outcomes and realistic future scope.



Key Outcomes

- ✓ Relational database designed
- ✓ CRUD operations implemented
- ✓ Integrity constraints applied
- ✓ Backend connected to MySQL
- ✓ Streamlit interface enables interaction

Future Scope

Authentication & role-based access
Seat selection and availability
Online payment integration
Advanced search / filtering
Deployment for real-world use

THANK YOU